

Matrix Multiplication.c Hashing (Linear Probing).c . Tree Traversals.c . Queue.c Stack Application (Postfix Example).c . Stack (Push Pop).c Linked List (Display).c Binary Search.c Linear Search.c . Array Display.cpp Fibonacci (Using Recursion).c

```
1 #include<stdio.h>
2 int fib(int n){
3     if(n<=1) return n;
4     return fib(n-1)+fib(n-2);
5 }
6 int main()
7 {
8     int i,n;
9     scanf("%d",&n);
10    for(i=0;i<n;i++)
11        printf("%d ",fib(i));
```

C:\DS LAB PROGRAMS\Fiboni x + v

6
0 1 1 2 3 5

Process exited after 2.929 seconds with return value 6
Press any key to continue . . . |

MERGE.C QUICK SORT.C HEAP SORT.C

```
1  #include <stdio.h>
2
3  void heapify(int a[],int n,int i){
4      int l=2*i+1,r=2*i+2,largest=i,t;
5      if(l<n&&a[l]>a[largest]) largest=l;
6      if(r<n&&a[r]>a[largest]) largest=r;
7      if(largest!=i){
8          t=a[i]; a[i]=a[largest]; a[largest]=t;
9          heapify(a,n,largest);
10     }
11 }
12
13 int main(){
14     int a[20],n,i,t;
15     scanf("%d",&n);
16     for(i=0;i<n;i++) scanf("%d",&a[i]);
17
18     for(i=n/2-1;i>=0;i--) heapify(a,n,i);
19     for(i=n-1;i>0;i--){
20         t=a[0]; a[0]=a[i]; a[i]=t;
21         heapify(a,i,0);
22     }
23
24     for(i=0;i<n;i++) printf("%d ",a[i]);
25 }
```

C:\DS LAB PROGRAMS\HEAP

```
5
12 3 8 5 1
1 3 5 8 12
-----
```

Process exited after 6.414 seconds with return value 0
Press any key to continue . . .

Matrix Multiplication.c Hashing (Linear Probing).c Tree Traversals.c Queue.c Stack Application (Postfix Example).c Stack (Push Pop).c Linked List (Display).c Binary Search.c Linear Search.c

```
1 #include<stdio.h>
2 int main()
3 {
4     int a[]={2,4,6,8,10},i,key=6;
5     for(i=0;i<5;i++)
6     {
7         if(a[i]==key){
8             printf("Found");
9             return 0;
10        }
11    }
12    printf("Not Found");
13 }
```

C:\DS LAB PROGRAMS\Linear × + ▾

Found

Process exited after 0.1254 seconds with return value 0
Press any key to continue . . . |

MERGE.C

```
1  #include <stdio.h>
2
3  void merge(int a[],int l,int m,int r){
4      int i=l,j=m+1,k=0,t[50];
5      while(i<=m&& j<=r)
6          t[k++]=(a[i]<a[j])?a[i++]:a[j++];
7      while(i<=m) t[k++]=a[i++];
8      while(j<=r) t[k++]=a[j++];
9      for(i=l,k=0;i<=r;i++,k++) a[i]=t[k];
10 }
11
12 void mergeSort(int a[],int l,int r){
13     if(l<r){
14         int m=(l+r)/2;
15         mergeSort(a,l,m);
16         mergeSort(a,m+1,r);
17         merge(a,l,m,r);
18     }
19 }
20
21 int main(){
22     int a[20],n,i;
23     scanf("%d",&n);
24     for(i=0;i<n;i++) scanf("%d",&a[i]);
25     mergeSort(a,0,n-1);
26     for(i=0;i<n;i++) printf("%d ",a[i]);
27 }
```

C:\DS LAB PROGRAMS\MERG x + v

```
5
4 5 4 5 7
4 4 5 5 7
-----
Process exited after 6.131 seconds with return value 0
Press any key to continue . . . |
```

Untitled1 . Factorial (Using Recursion).c Fibonacci (Without Recursion).c

```
1 #include<stdio.h>
2 int main()
3 {
4     int a=0,b=1,c,i,n;
5     scanf("%d",&n);
6     printf("%d %d ",a,b);
7     for(i=3;i<=n;i++){
8         c=a+b;
9         printf("%d ",c);
10        a=b;
11        b=c;
12    }
```

C:\DS LAB PROGRAMS\Fibon: X + v

5
0 1 1 2 3

Process exited after 6.333 seconds with return value 5
Press any key to continue . . . |

Matrix Multiplication.c Hashing (Linear Probing).c Tree Traversals.c Queue.c Stack Application (Postfix Example.c)

```
1 #include<stdio.h>
2 int main()
3 {
4     printf("Postfix: AB+");
```

C:\DS LAB PROGRAMS\Stack . X + v

Postfix: AB+

Process exited after 0.1223 seconds with return value 12

Press any key to continue . . . |

MERGE.C QUICK SHORT.C HEAP SORT.C AVL Tree (Simple BST Version).C . Breadth First Search (BFS).C Depth First Search (DFS).C 3. Dijkstra's Algorithm.C Prim's Algorithm.C Kruskal's Algorithm (Simple).C

```
1 #include <stdio.h>
2
3 int main()
4 {
5     printf("Kruskal's Algorithm uses:\n");
6     printf("1. Sort edges\n");
7     printf("2. Pick minimum edge\n");
8     printf("3. Avoid cycle\n");
9     printf("4. Repeat until MST formed\n");
10    return 0;
}
```

C:\DS LAB PROGRAMS\Kruska × + ▾

```
Kruskal's Algorithm uses:
1. Sort edges
2. Pick minimum edge
3. Avoid cycle
4. Repeat until MST formed
```

```
-----
Process exited after 0.09856 seconds with return value 0
Press any key to continue . . . |
```

MERGE.C QUICK SHORT.C HEAP SORT.C AVL Tree (Simple BST Version).C . Breadth First Search (BFS).C Depth First Search (DFS).C

```
1  #include <stdio.h>
2
3  int g[4][4]={{0,1,1,0},{1,0,1,1},{1,1,0,0},{0,1,0,0}},v[4];
4
5  void dfs(int s){
6      int i;
7      v[s]=1;
8      printf("%d ",s);
9      for(i=0;i<4;i++)
10         if(g[s][i]&&!v[i]) dfs(i);
11 }
12
13 int main(){
14     dfs(0);
15 }
```

C:\DS LAB PROGRAMS\Depth × + ▾

0 1 2 3

Process exited after 0.1248 seconds with return value 0
Press any key to continue . . . |

MERGE.C QUICK SORT.C HEAP SORT.C AVL Tree (Simple BST Version).C Breadth First Search (BFS).C Depth First Search (DFS).C 3. Dijkstra's Algorithm.C

```
1  #include <stdio.h>
2  #define INF 999
3
4  int main()
5  {
6      int g[3][3]={{0,4,1},{4,0,2},{1,2,0}};
7      int d[3]={0,INF,INF},v[3]={0},i,j,u,min;
8
9      for(i=0;i<3;i++){
10         min=INF;
11         for(j=0;j<3;j++){
12             if(!v[j]&&d[j]<min){min=d[j];u=j;}
13         }
14         v[u]=1;
15         for(j=0;j<3;j++){
16             if(g[u][j]&&d[j]>d[u]+g[u][j])
17                 d[j]=d[u]+g[u][j];
18         }
19     }
20     for(i=0;i<3;i++) printf("%d ",d[i]);
21 }
```

C:\DS LAB PROGRAMS\3. Dijk × + ▾
0 3 1

Process exited after 0.1193 seconds with return value 0
Press any key to continue . . . |

MERGE.C QUICK SHORT.C HEAP SORT.C AVL Tree (Simple BST Version).C . Breadth First Search (BFS).C

```
1  #include <stdio.h>
2  int main()
3  {
4      int g[4][4]={{0,1,1,0},{1,0,1,1},{1,1,0,0},{0,1,0,0}};
5      int q[10],f=0,r=0,v[4]={0},i,x;
6      q[r++]=0; v[0]=1;
7      while(f<r){
8          x=q[f++];
9          printf("%d ",x);
10         for(i=0;i<4;i++){
11             if(g[x][i]&&!v[i]){
12                 v[i]=1;
13                 q[r++]=i;
14             }
15         }
16     }
```

C:\DS LAB PROGRAMS\Breadth First Search (BFS).C

0 1 2 3

Process exited after 0.1175 seconds with return value 0
Press any key to continue . . .

Untitled1 . Factorial (Using Recursion).c

```
1  #include<stdio.h>
2  int fact(int n){
3      if(n==0) return 1;
4      return n*fact(n-1);
5  }
6  int main()
7  {
8      int n;
9      scanf("%d",&n);
10     printf("%d",fact(n));
```

C:\DS LAB PROGRAMS\ . Factc

5
120

Process exited after 2.155 seconds with return value 3
Press any key to continue . . . |

Matrix Multiplication.c Hashing (Linear Probing).c Tree Traversals.c Queue.c Stack Application (Postfix Example.c Stack (Push Pop).c Linked List (Display).c Binary Search.c Linear Search.c Array Display.cpp

```
1 #include<stdio.h>
2 int main()
3 {
4     int a[5]={10,20,30,40,50};
5     for(i=0;i<5;i++)
6         printf("%d ",a[i]);
```

C:\DS LAB PROGRAMS\ Array

10 20 30 40 50

Process exited after 0.1123 seconds with return value 0

Press any key to continue . . .

Untitled1 . Factorial (Using Recursion).c Fibonacci (Without Recursion).c Factorial (Without Recursion).c Odd or Even.c Matrix Multiplication.c

```
1 #include<stdio.h>
2 int main()
3 {
4     int a[2][2]={1,2},{3,4};
5     int b[2][2]={5,6},{7,8};
6     int c[2][2];
7     for(i=0;i<2;i++)
8     {
9         for(j=0;j<2;j++){
10             c[i][j]=0;
11             for(k=0;k<2;k++){
12                 c[i][j]+=a[i][k]*b[k][j];
13             }
14         }
15     }
16     for(i=0;i<2;i++){
17         for(j=0;j<2;j++){
18             printf("%d ",c[i][j]);
19         }
20         printf("\n");
21     }
22 }
```

C:\DS LAB PROGRAMS\Matrix

19 22
43 50

Process exited after 0.1395 seconds with return value 10
Press any key to continue . . .

Matrix Multiplication.c Hashing (Linear Probing).c Tree Traversals.c

```
1 #include<stdio.h>
2 int main()
3 {
4     printf("Inorder: 2 1 3\n");
5     printf("Preorder: 1 2 3\n");
6     printf("Postorder: 2 3 1");
7 }
```

C:\DS LAB PROGRAMS\ Tree' x + v

```
Inorder: 2 1 3
Preorder: 1 2 3
Postorder: 2 3 1
```

```
-----
Process exited after 0.1217 seconds with return value 16
Press any key to continue . . . |
```

Matrix Multiplication.c Hashing (Linear Probing).c Tree Traversals.c Queue.c

```
1 #include<stdio.h>
2 int main()
3 {
4     int q[5]={10,20,30},f=0,r=2,i;
5     for(i=f;i<=r;i++)
6         printf("%d ",q[i]);
```

C:\DS LAB PROGRAMS\ Queue × + ▾

10 20 30

Process exited after 0.104 seconds with return value 3
Press any key to continue . . . |

Matrix Multiplication.c Hashing (Linear Probing).c Tree Traversals.c Queue.c Stack Application (Postfix Example.c Stack (Push Pop).c

```
1 #include<stdio.h>
2 int main()
3 {
4     int stack[5],top=-1;
5     stack[++top]=10;
6     stack[++top]=20;
7     printf("Pop=%d",stack[top--]);
```

C:\DS LAB PROGRAMS\ Stack × + ▾

Pop=20

Process exited after 0.1202 seconds with return value 6
Press any key to continue . . . |

Matrix Multiplication.c Hashing (Linear Probing).c Tree Traversals.c Queue.c Stack Application (Postfix Example).c Stack (Push Pop).c Linked List (Display).c Binary Search.c

```
1 #include<stdio.h>
2 int main()
3 {
4     int a[]={2,4,6,8,10},l=0,h=4,m,key=8;
5     while(l<=h){
6         m=(l+h)/2;
7         if(a[m]==key){
8             printf("Found");
9             return 0;
10        }
11        if(key>a[m]) l=m+1;
12        else h=m-1;
13    }
14    printf("Not Found");
15 }
```

C:\DS LAB PROGRAMS\Binary x + v

Found

Process exited after 0.1066 seconds with return value 0
Press any key to continue . . .

Matrix Multiplication.c Hashing (Linear Probing).c

```
1 #include<stdio.h>
2 int main()
3 {
4     int hash[10]={0};
5     hash[5]=25;
6     printf("25 stored at index %d",5);
7 }
```

C:\DS LAB PROGRAMS\Hashir × + ▾

25 stored at index 5

Process exited after 0.1188 seconds with return value 20

Press any key to continue . . . |

Untitled1 . Factorial (Using Recursion).c Fibonacci (Without Recursion).c Factorial (Without Recursion).c Odd or Even.c

```
1  #include<stdio.h>
2  int main()
3  {
4      int n;
5      scanf("%d",&n);
6      if(n%2==0)
7          printf("Even");
8      else
9          printf("Odd");
```

C:\DS LAB PROGRAMS\Odd o × + ▾

8

Even

Process exited after 2.935 seconds with return value 4
Press any key to continue . . . |

MERGE.C QUICK SHORT.C HEAP SORT.C AVL Tree (Simple BST Version).C . Breadth First Search (BFS).C Depth First Search (DFS).C 3. Dijkstra's Algorithm.C Prim's Algorithm.C

```
1 #include <stdio.h>
2 #define INF 999
3
4 int main()
5 {
6     int g[3][3]={{0,2,3},{2,0,1},{3,1,0}};
7     int v[3]={1,0,0},i,j,min,a,b;
8
9     for(i=0;i<2;i++){
10         min=INF;
11         for(a=0;a<3;a++){
12             if(v[a])
13                 for(b=0;b<3;b++){
14                     if(!v[b]&&g[a][b]&&g[a][b]<min)
15                         min=g[a][b],j=b;
16                 }
17             printf("%d ",min);
18             v[j]=1;
19         }
20     }
```

C:\DS LAB PROGRAMS\Prim's

2 1

Process exited after 0.1 seconds with return value 0

Press any key to continue . . .

Untitled1 | Factorial (Using Recursion).c | Fibonacci (Without Recursion).c | Factorial (Without Recursion).c

```
1  #include<stdio.h>
2  int main()
3      int n,i,f=1;
4      scanf("%d",&n);
5      for(i=1;i<=n;i++)
6          f*=i;
7      printf("%d",f);
8  
```

C:\DS LAB PROGRAMS\Factor

5
120

Process exited after 1.872 seconds with return value 3
Press any key to continue . . .